

Zadanie: Backend Integration Test — QA Automation Engineer

Popis

Vašou úlohou je implementovať sadu automatizovaných integračných testov pre REST API pomocou **Java** a frameworku **Serenity BDD** s **REST Assured**.

Testované API: `https://jsonplaceholder.typicode.com`

Technické požiadavky

- **Java** 11 alebo vyššia
- **Gradle** (projekt musí byť buildovateľný)
- **Serenity BDD** + **serenity-rest-assured** + **JUnit 5** (alebo TestNG — na výber)
- Výsledky testov musia generovať **Serenity HTML report**

Odporúčaná štruktúra `build.gradle` **závislostí:**

```
plugins {  
    id 'java'  
    id 'net.serenity-bdd.serenity-gradle-plugin' version '4.1.20'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'net.serenity-bdd:serenity-core:4.1.20'  
    testImplementation 'net.serenity-bdd:serenity-rest-assured:4.1.20'  
    testImplementation 'net.serenity-bdd:serenity-junit5:4.1.20'  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'  
}  
  
test {  
    useJUnitPlatform()  
    finalizedBy aggregate  
}
```

Zadané testovacie scenáre

Endpoint: `/posts`

TC-01: Načítanie zoznamu príspevkov

- Pošli `GET /posts`
- Over, že HTTP status code je `200`
- Over, že odpoveď obsahuje `100` príspevkov
- Over, že každý príspevok obsahuje polia: `id`, `userId`, `title`, `body`

TC-02: Načítanie konkrétneho príspevku

- Pošli `GET /posts/1`
- Over, že HTTP status code je `200`
- Over, že `id` v odpovedi je `1`
- Over, že pole `title` nie je prázdne

TC-03: Načítanie neexistujúceho príspevku

- Pošli `GET /posts/9999`
- Over, že HTTP status code je `404`

TC-04: Vytvorenie nového príspevku

- Pošli `POST /posts` s telom:

```
{  
  "title": "Test Post",  
  "body": "This is a test body",  
  "userId": 1  
}
```

- Over, že HTTP status code je `201`
- Over, že odpoveď obsahuje vygenerované `id`
- Over, že `title` v odpovedi zodpovedá odoslanej hodnote

TC-05: Aktualizácia príspevku

- Pošli `PUT /posts/1` s telom:

```
{  
  "id": 1,  
  "title": "Updated Title",  
  "body": "Updated body",  
  "userId": 1  
}
```

- Over, že HTTP status code je `200`
- Over, že `title` v odpovedi je `"Updated Title"`

TC-06: Čiastočná aktualizácia príspevku

- Pošli `PATCH /posts/1` s telom:

```
{
  "title": "Patched Title"
}
```

- Over, že HTTP status code je `200`
- Over, že `title` v odpovedi je `"Patched Title"`

TC-07: Vymazanie príspevku

- Pošli `DELETE /posts/1`
 - Over, že HTTP status code je `200`
-

Endpoint: `/posts/{id}/comments`

TC-08: Načítanie komentárov k príspevku

- Pošli `GET /posts/1/comments`
 - Over, že HTTP status code je `200`
 - Over, že odpoveď nie je prázdny zoznam
 - Over, že každý komentár obsahuje pole `email` a `email` je vo validnom formáte (obsahuje `@`)
-

Endpoint: `/users`

TC-09: Načítanie používateľov a overenie štruktúry

- Pošli `GET /users`
- Over, že HTTP status code je `200`
- Over, že každý používateľ obsahuje vnorený objekt `address` s poľom `city`

TC-10: Filtrovanie používateľa podľa query parametra

- Pošli `GET /users?username=Bret`
 - Over, že HTTP status code je `200`
 - Over, že odpoveď obsahuje práve `1` používateľa
 - Over, že `username` vráteného používateľa je `"Bret"`
-

Optional (bonusové) požiadavky

Nasledujúce techniky sú voliteľné, ale ich použitie bude pozitívne hodnotené. Slúžia na posúdenie prístupu k automatizácii. Môžu byť aplikované vo všetkých relevantných test caseoch.

Práca s odpoveďou

- **JsonPath** — vyhľadávanie hodnôt v JSON odpovediach namiesto manuálneho parsovania

```
String firstTitle = response.jsonPath().getString("[0].title");
List<String> emails = response.jsonPath().getList("email");
```

- **Java Streams** — prehľadávanie a verifikácia zoznamov v odpovediach

```
boolean allValid = comments.stream()
    .map(Comment::getEmail)
    .allMatch(email -> email.contains("@"));
```

- **Serializácia/deserializácia na objekty** — mapovanie odpovede na POJO triedy namiesto práce s raw JSON

```
Post post = response.as(Post.class);
List<User> users = response.jsonPath().getList("", User.class);
```

Architektúra a Design Patterns

- **Dedenie / Base classes** — abstraktná `BaseApiTest` so zdieľaným setupom (base URI, common headers, RequestSpecification)

```
public abstract class BaseApiTest {
    @BeforeAll
    static void setUp() {
        RestAssured.baseURI = ConfigProvider.getBaseUrl();
    }
}
```

- **Builder pattern pre payloady** — `Post.builder().title("...").body("...").userId(1).build()` namiesto inline máp/JSON stringov
- **REST Assured Specifications** — reusable `RequestSpecification` a `ResponseSpecification`

```
RequestSpecification jsonRequest = new RequestSpecBuilder()
    .setContentType(ContentType.JSON)
    .setBaseUri(BASE_URL)
    .build();
```

- **Separácia vrstiev** — `@Step` actions oddelené od test tried, business logika oddelená od HTTP volaní

Pokročilé testovacie techniky

- **Parametrizované testy** — `@ParameterizedTest` napr. pre negatívne ID

```
@ParameterizedTest
@ValueSource(ints = {9999, 0, -1})
void shouldReturn404ForInvalidPostId(int id) { ... }
```

- **JSON Schema validation** — validácia štruktúry odpovede cez `matchesJsonSchemaInClasspath("post-schema.json")`
- **Soft assertions cez `assertAll`** — všetky asercie naraz, report ukáže všetky zlyhania

```
assertAll("Post response",
    () -> assertEquals(200, response.statusCode()),
    () -> assertNotNull(post.getId()),
    () -> assertEquals("Test Post", post.getTitle())
);
```

- **Custom Hamcrest matchery** — napr. `isValidEmail()`, čitateľnejšie ako inline conditions

```
assertThat(comment.getEmail(), isValidEmail());
```

Tooling a infraštruktúra

- **Konfigurácia pre rôzne environmenty** — `serenity.conf` s profilmi (`dev` , `test` , `prod`) prepínateľnými cez `-Denvironment=test`
- **Lombok** — `@Data` , `@Builder` pre POJO triedy (Post, User, Comment, Address)

```
@Data @Builder
public class Post { ... }
```

- **CI/CD pipeline** — `Jenkinsfile` / `.gitlab-ci.yml` / GitHub Actions workflow ktorý spustí testy a publikuje Serenity report
- **JUnit 5** `@Tag` — kategorizácia testov (`smoke` , `regression` , `negative`) s možnosťou spustenia podľa tagu

```
@Tag("smoke") @Test
void shouldGetAllPosts() { ... }
```

Code quality

- **Test data factory** — vlastná factory pre generovanie testovacích dát namiesto hardcoded hodnôt

```
Post post = PostFactory.validPost();
Post invalid = PostFactory.postWithMissingTitle();
```

- **Custom logging v Serenity reporte** — `Serenity.recordReportData()` pre attachnutie request/response pri zlyhaní
- **Rozšírené negatívne scenáre** — invalid payload pri POST (chýbajúce required polia), malformed JSON, neexistujúci endpoint

Hodnotené kritériá

Kritérium	Váha
Všetky testy sú funkčné a spustiteľné cez <code>./gradlew clean test</code>	Povinné
Správna štruktúra Serenity projektu (steps/actions oddelené od testov)	30 %
Čitateľnosť a pomenúvanie testov (Serenity report dáva zmysel)	20 %
Správne použitie asercií (nie len status kód)	20 %
Konfigurácia base URL cez <code>serenity.conf</code> alebo properties súbor	15 %
Ošetrovanie testovania negatívnych scenárov (TC-03)	15 %

Odovzdanie

- Git repozitár (GitHub/GitLab) s `README.md` popisujúcim ako spustiť testy
- Serenity HTML report priložený ako artefakt, alebo screenshots z reportu